

TorchKM: A GPU-Oriented Library for Kernel Learning and Model Selection

Yikai Zhang

YIKAI-ZHANG@UIOWA.EDU

*Department of Statistics and Actuarial Science
University of Iowa
Iowa City, IA 52242, USA*

Gaoxiang Jia

JIAGAOXIANG@GMAIL.COM

Individual Researcher

Jie Ding

DINGJ@UMN.EDU

*School of Statistics
University of Minnesota
Minneapolis, MN 55455, USA*

Boxiang Wang

BOXIANG-WANG@UIOWA.EDU

*Department of Statistics and Actuarial Science
University of Iowa
Iowa City, IA 52242, USA*

Editor: My editor

Abstract

TorchKM is an open-source library for kernel machines, including support vector machines, kernel logistic regression, and kernel quantile regression, with GPU acceleration. The library features a scikit-learn-style API and is designed to exploit GPU-friendly linear algebra, accelerating the full training and model-selection pipeline through intelligent reuse of matrix operations. **TorchKM** also supports the Nyström approximation for scalable kernel learning. Benchmarks show competitive predictive performance together with substantial speedups over standard baselines. Code and documentation are available at <https://github.com/YikaiZhang95/torchkm>, and the package can be readily installed via PyPI.

Keywords: cross-validation, GPU acceleration, kernel machines, Nyström approximation, support vector machines,

1 Introduction

Kernel machines, including support vector machines (SVMs, Cortes and Vapnik, 1995), have been a fundamental class of methods in machine learning for several decades. They are widely recognized for their strong predictive performance, a convex optimization framework with unique global solutions, and a rigorous theoretical foundation. Although deep neural networks currently dominate fields such as computer vision and natural language processing, kernel methods remain highly competitive in structured-data applications and continue to define the state-of-the-art in many fields, as evidenced by recent comprehensive evaluations in Cervantes et al. (2020).

In modern practice, however, kernel methods are often relegated to the “back seat” of the machine learning toolbox, and the primary barrier is computational cost. The performance of an SVM, for instance, is sensitive to hyperparameter choices, but identifying a good configuration can be prohibitively expensive. Classical implementations, such as LIB-SVM (Chang and Lin, 2011), scikit-learn (Pedregosa et al., 2011), and kernlab (Karatzoglou et al., 2004) follow a standard paradigm in which training and model selection are treated as separate stages. Although fitting a model at each tuning parameter can be made efficient through specialized algorithms, such as sequential minimal optimization (Platt, 1999), tuning is still typically carried out in an outer loop through repeated refitting over a parameter grid, often at substantial computation cost. As such, kernel methods are often under-tuned in practice, and thus may not achieve their full predictive potential.

One promising way to reduce computational cost is to exploit advances in modern computing hardware, in particular, the graphics processing unit (GPU) acceleration (Nickolls et al., 2008). For kernel machines, an important breakthrough in this direction was ThunderSVM (Wen et al., 2018), which demonstrated that SVM training can be substantially accelerated on GPUs and multi-core CPUs. TorchKM builds on this hardware-aware perspective, but targets a complementary bottleneck: in model-selection workflows, the dominant cost often arises not from a single model fit alone, but from repeated kernel-matrix computations and linear solves across cross-validation folds and tuning parameters.

We present **TorchKM**, an open-source, GPU-oriented library for efficient end-to-end kernel learning with integrated model selection. **TorchKM** supports a broad range of methods for both regression and classification, including quantile regression, logistic regression, SVM, distance-weighted discrimination (DWD). It provides a user-friendly, scikit-learn-style interface. Rather than directly porting an existing CPU solver to the GPU, **TorchKM** is built through the so-called algorithm-hardware co-design: its computational algorithms are designed to exploit GPU-friendly linear algebra. At its core are two key algorithmic ideas, an exact cross-validation formula and a spectral algorithm, which together speed up the full training and tuning pipeline and still produce *exact* solutions rather than approximations.

The key differences between **TorchKM** and the two representative libraries, **scikit-learn** and **ThunderSVM**, are summarized in Table 1. Notably, **TorchKM** features an integrated pipeline for joint training and tuning, which is complementary to ThunderSVM’s emphasis on fast GPU-accelerated SVM fitting. Section 2 describes the core features of **TorchKM**, including GPU acceleration, support for methods beyond SVM, probability calibration, and scalable approximation for large datasets. Section 3 then outlines the exact cross-validation and spectral algorithm that make this integration possible. Algorithmic details and further numerical results are provided in the appendix.

2 Package Overview and Workflow

TorchKM adopts a scikit-learn-style interface, so users already familiar with scikit-learn should find it easy to use. Taking the SVM as an example, the basic workflow is similar to that of `sklearn.svm.SVC`: users instantiate **TorchKMSVC**, call `fit` to train the SVM model, and then use `predict` for prediction.

The key difference from the standard `sklearn.svm.SVC` workflow is that **TorchKMSVC** accepts a sequence of tuning parameters through `Cs`, although a single value is also permitted.

Table 1: Comparisons representative kernel-learning libraries. ThunderSVM is known as a GPU-accelerated SVM library; TorchKM focuses on integrated model-selection pipeline.

Features	scikit-learn	ThunderSVM	TorchKM
GPU acceleration	✗	✓	✓
Integrated training and tuning	✗	✗	✓
Non-SVM losses	✓	✗	✓
Probability calibration	✓	✓	✓
Integrated Nyström	✓	✗	✓

In the standard scikit-learn workflow, model selection is typically handled by an outer loop over candidate values through functions such as `GridSearchCV` or `RandomizedSearchCV`, with a separate fit for each value. In `TorchKM`, by contrast, training and tuning are integrated into the algorithm itself rather than carried out through repeated external refits.

The basic usage of `TorchKMSVC` is demonstrated in the following code snippet.

```
import numpy as np
from sklearn.datasets import make_circles
from torchkm.estimators import TorchKMSVC
X, y = make_circles(1200, factor = 0.4, noise = 0.08, random_state=0)
# Candidate values of the tuning parameter
Cs = np.logspace(3, -3, num = 12)
# Use CUDA to enable GPU acceleration.
clf = TorchKMSVC(kernel = "rbf", Cs = Cs, device = 'cuda', probability = True)
clf.fit(X, y)
# Training error shown for illustration only; use held-out data in practice
clf.predict(X)
# Probability prediction via Platt scaling
clf.predict_proba(X)
# Nystrom approximation
clf.fit(X, y, low_rank = True)
```

Setting `device="cuda"` enables GPU computation, while `device="cpu"` uses the same workflow on a CPU when a GPU is not available. For SVMs, probability estimates are available through Platt scaling via `predict_proba`. By default, `low_rank=False`, in which case `TorchKM` gives exact SVM solutions; for larger data sets, users can opt to set `low_rank=True` to enable the Nyström method and obtain approximate solutions.

Although we use SVMs for illustration, the same interface extends to other methods, including `TorchKMDWD` for DWD (Marron et al., 2007; Wang and Zou, 2016, 2018), `TorchKMLogit` for kernel logistic regression, and `TorchKMKQR` for kernel quantile regression (Koenker and Hallock, 2001; Tang et al., 2026). Consequently, this shared API provides a consistent workflow across different kernel learning models.

```
# DWD
clf_DWD = TorchKMDWD(kernel = "rbf", Cs = Cs, cv = 5, device = 'cuda')
# Logistic regression
clf_Logit = TorchKMLogit(kernel = "rbf", Cs = Cs, cv = 5, device = 'cuda')
```

Table 2: Comparison of `scikit-learn`, `ThunderSVM`, and `TorchKM`, averaged over 50 independent runs, with standard errors in parentheses and best values in bold.

n	p	scikit-learn			ThunderSVM			TorchKM		
		obj. values		time (s)	obj. values		time (s)	obj. values		time (s)
10000	10	0.50	(0.01)	1131.4	1.11	(0.08)	38.0	0.47	(0.01)	23.3
	100	0.40	(0.02)	1974.7	1.04	(0.10)	62.7	0.37	(0.02)	17.9
	1000	0.52	(0.01)	14322.8	1.00	(0.07)	279.4	0.48	(0.01)	37.7
20000	10	0.49	(0.01)	3593.8	1.12	(0.08)	104.8	0.45	(0.01)	76.0
	100	0.29	(0.02)	6206.0	0.49	(0.11)	132.9	0.28	(0.01)	78.2
	1000	—	—	> 8h	0.58	(0.07)	580.8	0.47	(0.01)	129.3

3 Core Computational Algorithms and Benchmark Performance

In kernel learning, the full training-and-tuning pipeline is computationally expensive because both training split and tuning parameters change the linear system to be solved. In a naive K -fold search over L candidate values, this leads to roughly KL kernel solves, which require $O(KLn^3)$ operations. Due to those repeated cubic-cost operations, simply running the same algorithm on a GPU may yield only modest improvement. (See an example in Figure 1 of the supplement.) As such, the central idea behind `TorchKM` is to redesign the algorithm to shift the cubic-cost burden into matrix-vector operations. Hence the GPU’s parallel architecture is made fuller use to accelerate the whole training-and-tuning pipeline.

To achieve this, `TorchKM` employs two techniques, both of which retain exact solutions. First, an exact cross-validation formula represents each fold through a modified response vector while keeping the kernel matrix unchanged. Second, a spectral algorithm computes a single eigendecomposition of the kernel matrix and performs all subsequent updates for different tuning parameters via matrix-vector multiplications. Together, these strategies replace the aforementioned repeated cubic-cost operations with a single $O(n^3)$ decomposition followed by only $O(n^2)$ operations. Full details are provided in the supplement.

As shown in Table 2, `TorchKM` attains the lowest objective values and the shortest run times. All objective values were evaluated at the same tuning parameter under the same objective, equation (1) in the supplement. Time includes the full train-and-tune pipeline. In the largest setting with $n = 20,000$ and $p = 1,000$, `scikit-learn` did not even complete within the 8-hour limit, whereas `TorchKM` completed the full training-and-tuning task in only 129.3 seconds. `ThunderSVM` improves over the CPU-based workflow in run time. `TorchKM` successfully further improves the performance by integrating cross-validation into training process and reusing matrix computations across folds and tuning parameters.

4 Conclusion

We introduced `TorchKM`, an open-source library for kernel learning with integrated model selection. `TorchKM` combines the exact cross-validation formula and spectral algorithm within `scikit-learn`-style interface for SVM, DWD, logistic, and quantile regression. Consequently, `TorchKM` enriches the software toolbox for kernel learning and supports its continued use in modern structured-data applications.

Supplement

Appendix A. Core Algorithm

In this appendix, we introduce the core algorithm behind our library **TorchKM**:

A.1 Kernel SVM

In this section, we use the kernel SVM as an illustration. Given the training data, $\{(y_i, \mathbf{x}_i)\}_{i=1}^n$, the kernel SVM can be formulated as:

$$(\hat{\alpha}, \hat{\beta}_0) = \underset{\alpha \in \mathbb{R}^n, \beta_0 \in \mathbb{R}}{\operatorname{argmin}} \left[\frac{1}{n} \sum_{i=1}^n \left(1 - y_i (\mathbf{K}_i^\top \alpha + \beta_0) \right)_+ + \lambda \alpha^\top \mathbf{K} \alpha \right], \quad (1)$$

where $(1 - u)_+ = \max\{1 - u, 0\}$ is the non-differentiable SVM hinge loss and $\lambda > 0$ is a tuning parameter.

Remark 1 Note that *scikit-learn*'s *SVC* uses C as the tuning parameter from the *libsvm* and *liblinear* convention. Thus the objective is of the form:

$$\min_{\alpha, \beta_0} \left[C \sum_{i=1}^n \left(1 - y_i (\mathbf{K}_i^\top \alpha + \beta_0) \right)_+ + \frac{1}{2} \alpha^\top \mathbf{K} \alpha \right],$$

where C plays the inverse role of the regularization strength; C and λ are in one-to-one correspondence through $C = \frac{1}{2n\lambda}$.

To solve the optimization problem (1), **TorchKM** implements the finite smoothing algorithm (Wang and Zou, 2022) which transforms the hinge loss function into a sequence of smooth optimization problems with a δ -smoothed hinge loss,

$$L_\delta(u) = \begin{cases} 1 - u & u \leq 1 - \delta, \\ \frac{1}{4\delta} [u - (1 + \delta)]^2 & 1 - \delta < u < 1 + \delta, \\ 0 & u \geq 1 + \delta, \end{cases}$$

and obtain the exact SVM solution.

The smoothed problem is then solved using the proximal gradient descent. The update formula is given by:

$$\begin{pmatrix} \beta_0^{(k+1)} \\ \alpha^{(k+1)} \end{pmatrix} - \begin{pmatrix} \beta_0^{(k)} \\ \alpha^{(k)} \end{pmatrix} = -\mathbf{H}_\lambda^{-1}(\mathbf{K}) \begin{pmatrix} \mathbf{1}^\top \mathbf{z}^{(k)} \\ \mathbf{K} \mathbf{z}^{(k)} + 2\lambda \mathbf{K} \alpha^{(k)} \end{pmatrix},$$

where $\mathbf{H}_\lambda(\mathbf{K}) = 2\lambda \mathbf{K} + \frac{1}{n\kappa} \mathbf{K} \mathbf{K}$ and $\mathbf{z}^{(k)} = (z_1, z_2, \dots, z_n)^\top$ with each $z_i = y_i L'_\delta[y_i(\beta_0^{(k)} + \mathbf{K}_i \alpha^{(k)})]/n$.

From the above update formula, we see that the main computational bottleneck in training kernel SVMs is computing $\mathbf{H}_\lambda^{-1}(\mathbf{K})$, which has $\mathcal{O}(n^3)$ complexity and remains costly with GPU acceleration. Additionally, cross-validation for model selection and computing solutions along the λ path requires repeated model fitting.

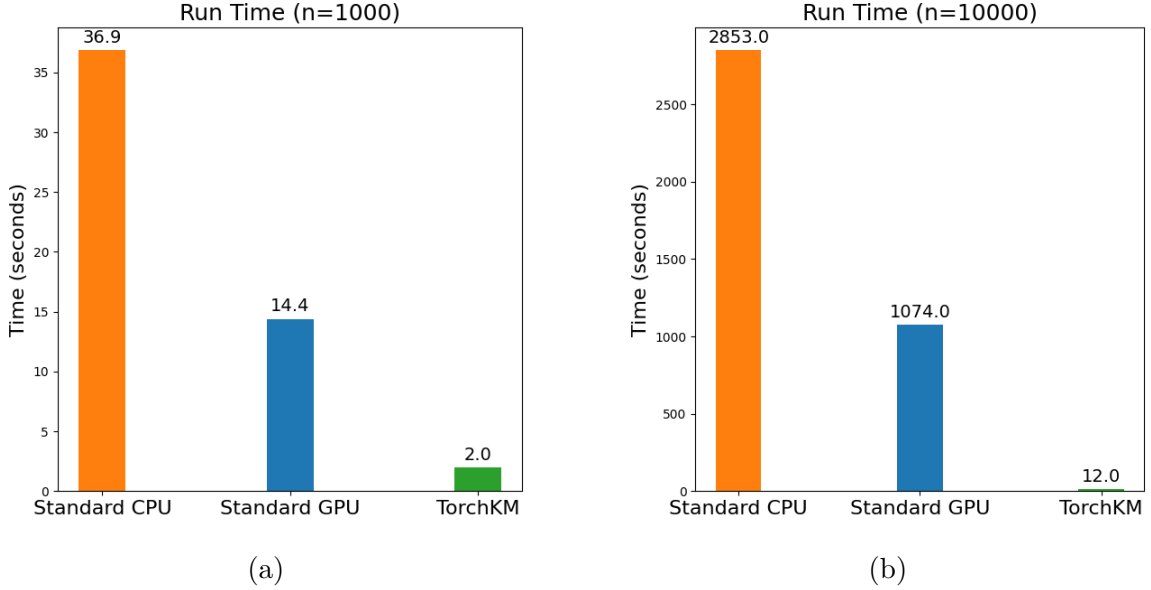


Figure 1: Run time for standard SVM training and tuning under CPU and GPU computation on simulated data sets. The standard SVM was fitted by proximal gradient descent, and model selection was performed by 5-fold cross-validation over 50 regularization parameter values. Panel (a) shows results for a data set of size $n = 1,000$, and panel (b) shows results for a data set of size $n = 10,000$. Lower run time indicates better computational performance. Because model fitting is repeated across cross-validation folds and along the regularization path, GPU offloading alone provides only limited speedup.

To illustrate the benefit of algorithm-hardware co-design, we conducted a simulation study comparing the full SVM training-and-tuning pipeline under three implementations: a standard proximal-gradient solver on CPU, the same solver on GPU, and **TorchKM**. We considered simulated data sets which are generated from a mixture-of-Gaussians model in Hastie et al. (2009). Let $\mu_+, \mu_- \in \mathbb{R}^p$ have entries equal to μ on disjoint halves of their coordinates and zero elsewhere. For each replicate, we sample centers $\mu_k^+ \sim \mathcal{N}(\mu_+, I)$ and $\mu_k^- \sim \mathcal{N}(\mu_-, I)$ for $k = 1, \dots, 10$. Positive examples are then drawn from the equal-weight mixture $\frac{1}{10} \sum_{k=1}^{10} \mathcal{N}(\mu_k^+, \sigma I)$, and negative examples from the analogous mixture on the μ_k^- side. We set $\mu = 2$, $\sigma = 3$, and consider $n \in \{1000, 10000\}$ with $p = 10$. For each data set, we computed the solution path over 50 regularization parameters and performed model selection using 5-fold cross-validation. As shown in Figure 1, GPU acceleration of the standard solver reduces run time, but the repeated computations across cross-validation folds and tuning parameters still remain a major bottleneck. In contrast, **TorchKM** achieves substantially larger speedups by reusing matrix computations across the training-and-tuning pipeline. When $n = 10,000$, **TorchKM** is more than two orders of magnitude faster than the standard CPU implementation and nearly two orders of magnitude faster than the standard GPU implementation. These results show that simply offloading computation to the GPU is not sufficient; substantial acceleration requires algorithms designed to exploit GPU-friendly linear algebra. Hardware and software details are provided in Section B.1.

To address these challenges, we introduce two strategies in the following sections: an efficient exact cross-validation method and a spectral algorithm that avoids repeated fitting when computing the solution path. Although the discussion above focuses on kernel SVM, we use it as a representative example. For kernel methods with smooth losses, since the smoothing step is unnecessary, the corresponding derivations are more direct.

A.2 Exact Cross-Validation Formula

Traditional cross-validation methods require the model to be retrained separately for each fold. This repeated fitting is computationally expensive and can even become infeasible when the dataset is large or when leave-one-out cross-validation is required. In order to make cross-validation practical for large-scale kernel classification, Wang and Zou (2022) introduced a strategy that avoids repeatedly retraining models. Given a loss function $L(\cdot)$. Let $[v]$ denote the left-out fold with m observations and define $\mathbf{K}_{[v]} \in \mathbb{R}^{(n-m) \times (n-m)}$ as the corresponding kernel matrix with the i -th rows and columns removed for $i \in [v]$. For each tuning parameter λ , the cross-validation solution obtained by removing the set $[v]$ from the training set is given by:

$$(\hat{\alpha}^{[-v]}, \hat{\beta}_0^{[-v]}) = \underset{\alpha \in \mathbb{R}^{n-m}, \beta_0 \in \mathbb{R}}{\operatorname{argmin}} \left[\frac{1}{n-m} \sum_{j \notin [v]} L(y_j(\mathbf{K}_{[v]}^\top \alpha + \beta_0)) + \lambda \alpha^\top \mathbf{K}_{[v]} \alpha \right].$$

Instead of refitting a separate model for each fold, the optimization is reformulated so that the same kernel matrix can be reused, with only minor modifications to the response vector. In particular, define $\tilde{\mathbf{y}}^{[v]}$ by letting $\tilde{y}_j^{[v]} = 0$ for $j \in v$ and $\tilde{y}_j^{[v]} = y_j$ for all $j \notin [v]$, and then by Lemma 2 of Wang and Zou (2022), it holds that

$$(\tilde{\alpha}^{[-v]}, \tilde{\beta}_0^{[-v]}) = \underset{\alpha \in \mathbb{R}^n, \beta_0 \in \mathbb{R}}{\operatorname{argmin}} \left[\frac{1}{n-m} \sum_{j=1}^n L(\tilde{y}_j^{[v]}(\mathbf{K}_j^\top \alpha + \beta_0)) + \lambda \alpha^\top \mathbf{K} \alpha \right].$$

and $\hat{\alpha}^{[-v]}$ can be obtained from $\tilde{\alpha}^{[-v]}$ by removing the entries corresponding to the held-out observations. Consequently, the exact cross-validation formula allows us to work with the same kernel matrix and slightly modified $\tilde{y}_j^{[v]}$. We compute and store the inversion of $\mathbf{H}_\lambda(\mathbf{K})$ only once, avoiding the need to invert the matrix V times.

A.3 Spectral Algorithm

Even with the exact cross-validation formula, we still need to solve the optimization problem along the solution path for each tuning parameter λ . Recall that the kernel SVM training updates take the following form:

$$\begin{pmatrix} \beta_0^{(k+1)} \\ \alpha^{(k+1)} \end{pmatrix} - \begin{pmatrix} \beta_0^{(k)} \\ \alpha^{(k)} \end{pmatrix} = -\mathbf{H}_\lambda^{-1}(\mathbf{K}) \mathbf{d}^{(k)}, \quad \text{where } \mathbf{d}^{(k)} = \begin{pmatrix} \mathbf{1}^\top \mathbf{z}^{(k)} \\ \mathbf{K} \mathbf{z}^{(k)} + 2\lambda \mathbf{K} \alpha^{(k)} \end{pmatrix}$$

and $\mathbf{H}_\lambda(\mathbf{K}) = 2\lambda \mathbf{K} + \frac{1}{n\kappa} \mathbf{K} \mathbf{K}$. Each step would require a new inversion of $\mathbf{H}_\lambda(\mathbf{K})$ for every λ , whose complexity is $O(n^3)$.

Algorithm 1 cvksvm in TorchKM

Require: $\mathbf{y}, \mathbf{K}$, and λ sequence

- 1: Initialize δ . The smooth function is L_δ . Let $\kappa = 2\delta$.
 - 2: Initialize $\tilde{\boldsymbol{\alpha}}^{[-i]} = \mathbf{0}$ for $i = 1, \dots, n$.
 - 3: Compute eigendecomposition $\mathbf{K} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$
 - 4: **for** each tuning parameter λ_ℓ , $\ell = 1, 2, \dots, L$, **do**
 - 5: **repeat**
 - 6: Compute $\boldsymbol{\Pi}_{\lambda_\ell} = 2\lambda_\ell\boldsymbol{\Lambda} + 1/(n\kappa)\boldsymbol{\Lambda}\boldsymbol{\Lambda}$.
 - 7: Compute $\mathbf{v} = \mathbf{U}\boldsymbol{\Lambda}\boldsymbol{\Pi}_{\lambda_\ell}^{-1}\mathbf{U}^\top\mathbf{1}$ and $g = 1/(n\mathbf{1}^\top\mathbf{U}\boldsymbol{\Lambda}\boldsymbol{\Pi}_{\lambda_\ell}^{-1}\boldsymbol{\Lambda}\mathbf{U}^\top\mathbf{1})$.
 - 8: **for** $i = 1, \dots, n$ **do**
 - 9: Let $\tilde{y}_j^{[-i]} = y_j$ if $j \neq i$, and $\tilde{y}_i^{[-i]} = 0$.
 - 10: Set $(\bar{\boldsymbol{\alpha}}, \bar{\beta}_0) \leftarrow (\tilde{\boldsymbol{\alpha}}^{[-i]}, \tilde{\beta}_0^{[-i]})$ and $(\boldsymbol{\alpha}', \beta'_0) \leftarrow (\tilde{\boldsymbol{\alpha}}^{[-i]}, \tilde{\beta}_0^{[-i]})$.
 - 11: Let $r = 1$.
 - 12: **repeat**
 - 13: Compute $r' = \frac{1+\sqrt{1+4r^2}}{2}$.
 - 14: Update
 - $(\bar{\boldsymbol{\alpha}}, \bar{\beta}_0) \leftarrow (\tilde{\boldsymbol{\alpha}}^{[-i]}, \tilde{\beta}_0^{[-i]}) + \frac{r-1}{r'} \left\{ (\tilde{\boldsymbol{\alpha}}^{[-i]}, \tilde{\beta}_0^{[-i]}) - (\boldsymbol{\alpha}', \beta'_0) \right\}.$
 - 15: Let $\bar{\mathbf{z}} = (\bar{z}_1, \dots, \bar{z}_n)^\top$, with $\bar{z}_j = \tilde{y}_j^{[-i]} L'_\delta[\tilde{y}_j^{[-i]}(\mathbf{K}_j^\top \bar{\boldsymbol{\alpha}} + \bar{\beta}_0)]/n$.
 - 16: From right to left avoiding matrix multiplications, compute
 - $\Delta\boldsymbol{\alpha} = g(-(\mathbf{1}^\top \bar{\mathbf{z}})\mathbf{v} + \mathbf{v}\mathbf{v}^\top \mathbf{K}(\bar{\mathbf{z}} + 2\lambda_\ell \bar{\boldsymbol{\alpha}})) + \mathbf{U}\boldsymbol{\Lambda}\boldsymbol{\Pi}_{\lambda_\ell}^{-1}\mathbf{U}^\top(\bar{\mathbf{z}} + 2\lambda_\ell \bar{\boldsymbol{\alpha}}).$
 - 17: Compute $\Delta\beta_0 = g(\mathbf{1}^\top \bar{\mathbf{z}}) - \mathbf{v}^\top \mathbf{K}(\bar{\mathbf{z}} + 2\lambda_\ell \bar{\boldsymbol{\alpha}})$
 - 18: Update $(\boldsymbol{\alpha}', \beta'_0) \leftarrow (\tilde{\boldsymbol{\alpha}}^{[-i]}, \tilde{\beta}_0^{[-i]})$.
 - 19: Update $(\tilde{\boldsymbol{\alpha}}^{[-i]}, \tilde{\beta}_0^{[-i]}) \leftarrow (\bar{\boldsymbol{\alpha}} - \Delta\boldsymbol{\alpha}, \bar{\beta}_0 - \Delta\beta_0)$.
 - 20: Update $r \leftarrow r'$.
 - 21: **until** the convergence condition is met.
 - 22: **end for**
 - 23: Shrink $\delta = \eta\delta$, where $\eta = 0.125$. Update L_δ and $\kappa = 2\delta$.
 - 24: **until** the KKT conditions of all SVM models are satisfied.
 - 25: **end for**
-

To address this, TorchKM applies a spectral algorithm: compute the eigendecomposition $\mathbf{K} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$ first. For each λ , define $\boldsymbol{\Pi}_\lambda = 2\lambda\boldsymbol{\Lambda} + \frac{1}{n\kappa}\boldsymbol{\Lambda}\boldsymbol{\Lambda}$. Then $\mathbf{H}_\lambda^{-1}$ can be written as:

$$\mathbf{H}_\lambda^{-1}(\mathbf{K}) = g \begin{pmatrix} 1 \\ -\mathbf{v} \end{pmatrix} \begin{pmatrix} 1 & -\mathbf{v}^\top \end{pmatrix} + \begin{pmatrix} 0 & \mathbf{0}^\top \\ \mathbf{0} & \mathbf{U}\boldsymbol{\Pi}_\lambda^{-1}\mathbf{U}^\top \end{pmatrix},$$

where $\mathbf{v} = \mathbf{U}\boldsymbol{\Lambda}\boldsymbol{\Pi}_\lambda^{-1}\mathbf{U}^\top\mathbf{1}$, and $g = 1/(n\mathbf{1}^\top\mathbf{U}\boldsymbol{\Lambda}\boldsymbol{\Pi}_\lambda^{-1}\boldsymbol{\Lambda}\mathbf{U}^\top\mathbf{1})$. At the k th iteration, we compute $\mathbf{H}_\lambda^{-1}(\mathbf{K})\mathbf{d}^{(k)}$ from right to left with only $O(n^2)$ complexity.

Consequently, after a single $O(n^3)$ eigendecomposition of $\mathbf{K}$, the subsequent computations along the λ -path reduce to $O(n^2)$ matrix–vector multiplications. This structure is well suited to GPU parallelism and is a key reason for the computational efficiency of `TorchKM`.

A.4 The `TorchKM` Algorithm

With the exact cross-validation formula and the spectral algorithm, we implement the `TorchKM` algorithm. For illustration, we present the leave-one-out cross-validation version for kernel SVM over a sequence of tuning parameters $\lambda_1, \lambda_2, \dots, \lambda_L$. To further accelerate computation along the regularization path, we use warm starts: the solution obtained at λ_ℓ is used as the initial value for solving the problem at $\lambda_{\ell+1}$. We also incorporate Nesterov’s acceleration (Nesterov, 1983) to speed up convergence.

A.5 Nyström kernel approximation

`TorchKM` also supports the Nyström approximation for large-scale problems. Working directly with a full $n \times n$ kernel matrix quickly becomes infeasible when n is large. The Nyström method addresses this issue by constructing a compressed representation of the kernel using only a subset of the training points. `TorchKM` adopts a customized Nyström kernel approximation framework that avoids recomputing the approximation for every regularization parameter and every cross-validation fold, as is typically required in conventional Nyström methods. `TorchKM` can perform a single unified Nyström approximation of $\mathbf{K}$ outside the λ -path and cross-validation loops, making model training efficient.

A.6 Platt Scaling plot

`TorchKM` also provides calibrated probability estimates in addition to class predictions. Platt scaling is a probability calibration method that transforms a model’s raw decision scores into estimated class probabilities by fitting a sigmoid function. Its main purpose is to improve the interpretability and reliability of classifier. In this section, we used the same binary classification setting as in the previous section. The training set consisted of $n = 10000$ observations, and the test set consisted of 1000 observations, each with $p = 10$ predictors. The RBF-kernel SVM was fitted over a sequence of 50 regularization parameter values. The Platt calibration plot was then used to evaluate how well the calibrated probabilities align with the observed outcome frequencies. We evaluated the calibration performance on an independent test set using reliability curve with the expected calibration error (ECE), and the Brier score. Figure 2 shows the reliability curve for Platt-calibrated probabilities on the independent test set. The dashed line denotes perfect calibration. The curve remains close to the identity line overall, with ECE of 0.044 and Brier score of 0.130, which indicates good overall calibration.

Appendix B. Additional Experiment Results

B.1 Experimental Setup

Hardware and software environment. All timing experiments reported in Tables 2–4 were carried out on a single workstation equipped with one NVIDIA L40s GPU with 48 GB

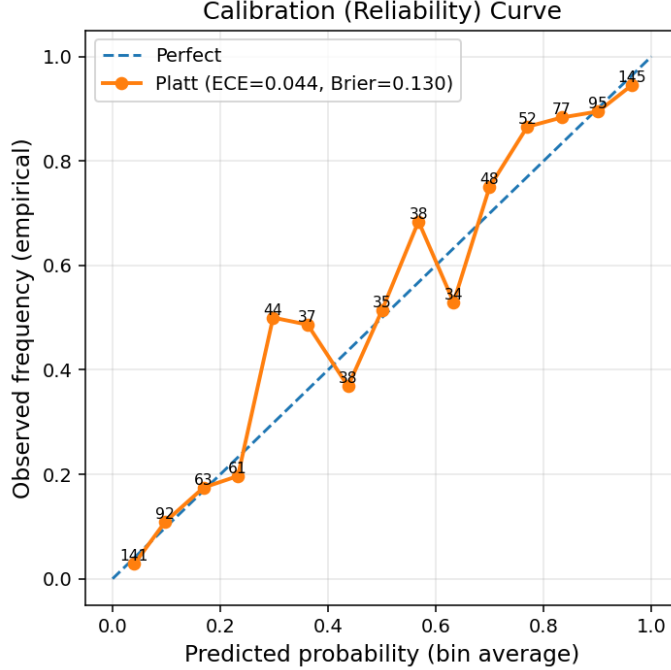


Figure 2: Reliability curve for Platt-calibrated probabilities on the independent test set. The plotted curve compares observed frequencies with predicted probabilities, and the dashed 45° line denotes perfect calibration. Curves closer to the diagonal indicate better calibration. The expected calibration error (ECE) is 0.044 and the Brier score is 0.130; smaller values indicate better calibration, suggesting good overall agreement between predicted probabilities and observed frequencies.

of memory, an AMD EPYC 9334 (32-Core) CPU, and 768 GB of system RAM, running Ubuntu 22.04 with CUDA 12.1. **TorchKM** was executed under Python 3.11 with PyTorch 2.4.1 (CUDA build). The **scikit-learn** baseline used **scikit-learn** 1.1.3 together with NumPy 1.25.2 and SciPy 1.9.3 on the CPU. **ThunderSVM** was built from the official release 0.3.4 against CUDA 12.1 and run on the same GPU as **TorchKM**. The neural-network baseline in Tables 3–4 was implemented in PyTorch 2.4.1 and trained on the same GPU. All run times are wall-clock times measured end-to-end for the full training-and-tuning pipeline, averaged over 50 runs in Table 2 and 10 runs in Tables 3–4.

Tuning-parameter grid and cross-validation. For all libraries, we used a grid of 50 candidate regularization values, spaced log-uniformly over $[-3, 3]$, corresponding to

$$C \in [10^{-3}, 10^3]$$

under the **scikit-learn**/LIBSVM parameterization. Equivalently, one may use the λ -parameterization by converting each value of C via

$$C = \frac{1}{2n\lambda}.$$

For **TorchKM**, model selection in Tables 2–4 was performed using 10-fold cross-validation. The competing libraries were also tuned using 10-fold cross-validation via **GridSearchCV** or **cross_val_score**.

Benchmark data. Tables 3 and 4 use standard benchmark classification datasets from the LIBSVM repository maintained at National Taiwan University, available at <https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/>. The specific files we used are:

- **a7a, a8a, a9a** (UCI Adult):
<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary/a7a>,
<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary/a8a>,
<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary/a9a>.
- **w7a, w8a** (Web Page):
<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary/w7a>,
<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary/w8a>.
- **ijcnn1**:
<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary/ijcnn1.bz2>.
- **covtype.binary**:
<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary/covtype.libsvm.binary.scale.bz2>.
- **MNIST8m (4 vs. 6)**: extracted as the subset of digits 4 and 6 from `mnist8m.scale.xz` at
<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/multiclass/mnist8m.scale.xz>; see also <https://leon.bottou.org/projects/infimnist>.

Each data set was used in its standard LIBSVM-format release without further preprocessing beyond feature scaling to $[-1, 1]$ where provided.

B.2 Nyström kernel SVM on benchmarks

In this section, we compared the neural-network baseline, **TorchKM**, and **ThunderSVM** on three benchmark datasets in terms of classification accuracy and run time. Specifically, we use a lightweight 1D convolutional neural network (CNN) consisting of two convolutional layers with ReLU activations, followed by an adaptive average pooling layer. The benchmark datasets used in this experiment are **a7a**, **a8a**, and **w7a**, which contain 16,100, 22,696, and 24,692 samples, respectively. Table 3 shows that **TorchKM** consistently improves predictive accuracy while significantly reducing run time, demonstrating its advantage as an efficient and effective kernel learning approach.

For the larger-scale evaluation, we used five benchmark datasets: **a9a**, **w8a**, **ijcnn**, **covtype**, and **MNIST8m (4 vs 6)**, which contain 32,561, 49,749, 49,990, 581,012, and 1,270,178 samples, respectively. These datasets were selected to extend the comparison to larger classification problems and to examine the scalability of the solvers. We evaluated both the predictive performance and the tuning efficiency of the neural network, **TorchKM** with Nyström approximation, and Nyström implemented in **scikit-learn**.

Table 3: Classification accuracy and run time for a neural network baseline, **TorchKM**, and **ThunderSVM** on the **a7a**, **a8a**, and **w7a** benchmark classification data sets. Numbers in parentheses give the sample size of each data set. Higher accuracy and lower run time indicate better performance. Across the three data sets, **TorchKM** attains the best accuracy and the shortest run time.

Data (size)	Neural-network		TorchKM		ThunderSVM	
	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)
a7a (16,100)	0.810	134.78	0.830	48.53	0.828	108.50
a8a (22,696)	0.814	191.44	0.833	106.92	0.829	154.28
w7a (24,692)	0.971	475.69	0.971	135.06	0.971	203.11

Table 4: Classification accuracy (Acc) and run time (Time, in second) for a neural-network baseline, **cvknyssvm** in **TorchKM**, and the Nyström approximation in **scikit-learn** on the **a9a**, **w8a**, **ijcnn**, **covtype**, and **MNIST8m** (4 vs 6) benchmark classification data sets. Numbers in parentheses give the sample size of each data set. Higher accuracy and lower run time indicate better performance. Across all five data sets, **TorchKM** attains the highest accuracy and the shortest run time, with the largest gain appearing on **MNIST8m** (4 vs 6).

Data (size)	Neural-network		TorchKM		Nyström (sklearn)	
	Acc	Time (s)	Acc	Time (s)	Acc	Time (s)
a9a (32,561)	0.805	270.55	0.851	17.86	0.849	205.83
w8a (49,749)	0.971	475.69	0.981	20.78	0.976	275.37
ijcnn1 (49,990)	0.916	316.85	0.982	27.42	0.977	275.49
covtype (581,012)	0.805	3249.55	0.807	31.11	0.786	2269.35
MNIST8m (1,270,178)	0.978	14330.60	0.997	64.65	0.996	5189.03

Table 4 presents the comparison of these solvers on the larger benchmark datasets in terms of accuracy and run time. The improvement of **TorchKM** becomes even more pronounced on **MNIST8m** (4 vs 6), where it obtains the highest accuracy of 0.997 in just 64.647 seconds, compared with 0.978 in 14330.60 seconds for the neural network and 0.996 in 5189.03 seconds for **scikit-learn** with Nyström. These results demonstrate that **TorchKM** not only delivers consistently superior accuracy, but also scales far more efficiently on large-scale datasets.

References

- Jair Cervantes, Farid Garcia-Lamont, Lisbeth Rodríguez-Mazahua, and Asdrubal Lopez. A comprehensive survey on support vector machine classification: Applications, challenges and trends. *Neurocomputing*, 408:189–215, 2020.
- Chih-Chung Chang and Chih-Jen Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 2(3):1–27, 2011.
- Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2nd edition, 2009.
- Alexandros Karatzoglou, Alexandros Smola, Kurt Hornik, and Achim Zeileis. kernlab—an S4 package for kernel methods in R. *Journal of Statistical Software*, 11:1–20, 2004.
- Roger Koenker and Kevin F Hallock. Quantile regression. *Journal of Economic Perspectives*, 15(4):143–156, 2001.
- James Stephen Marron, Michael J Todd, and Jeongyoun Ahn. Distance-weighted discrimination. *Journal of the American Statistical Association*, 102(480):1267–1271, 2007.
- Yurii Nesterov. A method for solving the convex programming problem with convergence rate $O(1/k^2)$. 269(3):543–547, 1983.
- John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. Scalable parallel programming with CUDA: Is CUDA the parallel programming model that application developers have been waiting for? *Queue*, 6(2):40–53, 2008.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- John Platt. Fast training of support vector machines using sequential minimal optimization. *Advances in Kernel Methods: Support Vector Learning*, pages 185–208, 1999.
- Qian Tang, Yuwen Gu, and Boxiang Wang. fastkqr: A fast algorithm for kernel quantile regression. *Journal of Computational and Graphical Statistics*, 35(1):395–405, 2026.
- Boxiang Wang and Hui Zou. Sparse distance weighted discrimination. *Journal of Computational and Graphical Statistics*, 25(3):826–838, 2016.
- Boxiang Wang and Hui Zou. Another look at distance-weighted discrimination. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 80(1):177–198, 2018.
- Boxiang Wang and Hui Zou. Fast and exact leave-one-out analysis of large-margin classifiers. *Technometrics*, 64(3):291–298, 2022.
- Zeyi Wen, Jiashuai Shi, Qinbin Li, Bingsheng He, and Jian Chen. ThunderSVM: A fast SVM library on GPUs and CPUs. *Journal of Machine Learning Research*, 19(21):1–5, 2018.